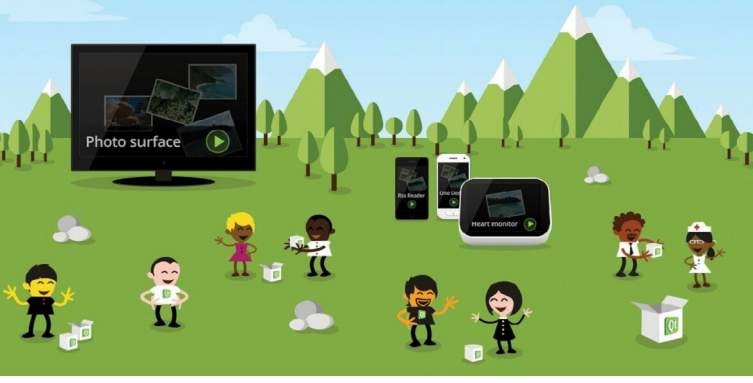
Qt5: Let's Learn Some Theory

This article marks the beginning of a new column on Qt5 programming, which is a platform-independent application framework. It is widely used for developing application software that can be run on diverse hardware and software platforms with practically no change in the code base.



Qt (pronounced Cute) has a long and illustrious history. Originally developed by a company called Trolltech, Nokia took over the reins of this application framework before handing it over to Digia. Qt is no longer just a set of libraries for C++; it's an entire application framework. It's one of the very few pieces of software you'll find that's triple licensed—you have a choice between LGPL-2.1 and GPLv3 if you want to use the open source version of it, or you can get a Qt Commercial License if you want to use the proprietary version.

Here's one of the greatest misconceptions about Qt— it's not a GUI library. Well, it is, but GUIs are only part of what Qt can do. In fact, in the course of this series of articles, we're going to develop a network-based application using Qt, and the server is going to be a CLI application. Qt is going to introduce you to a completely new programming paradigm, and once you've finished your first Qt application, you will not want to go back to using vanilla C++, ever.

A primer on Qt's architecture

There's a lot of difference between the code that you write when programming with Qt, and the code that actually gets compiled.

Qt's primary language is C++. Bindings to other languages are available, but if you're just starting out with Qt, it's better to start with C++, because there are a bunch of concepts you'll need to understand and it'll be much easier to understand them if you're using C++.

But Qt itself extends the entire C++ language and makes it much more flexible and powerful, thanks to the unique pre-processing and code-generation that it does before compiling any C++ code.

Let's get started.

Part 1: Signals and slots

When you write standard C++ code, you have three access modifiers available for class members: there are *private* members, the visibility of which is restricted to other members of the class only; the *public* members, which can be seen from outside the object, and the *protected* members, which are hidden from public view but can be inherited.

Qt implements a programming paradigm called the